Developer Documentation

Bayarjargal Munkhsaikhan (FPQV85)

1. Introduction

The C program is a basic Song Database Management System. It allows user to perform operations such as adding new songs, listing all entries, searching for songs, deleting songs, saving the database to a file, and exiting the program. The program uses a linked list data structure to store the songs.

2. Modules

Main Module (main function):

- Implements the user interface and menu system.
- Calls functions based on user input.

Database Module:

- Defines the song structure to represent a song.
- Implements functions to load, add, save, list, search, delete, and free the song database.

3. Data Structures:

Struct song:

- Represents a song with attributes like title, artist, album, release year, genre, and song length.
- Contains a pointer to the next song in the linked list.

4. Algorithms:

Load Database (loadDatabase):

- Reads song entries from a file and creates a linked list of songs.

Add Song (addSong):

- Dynamically allocates memory for a new song.
- Takes user input for song details.
- Adjusts song length if seconds exceed 60.

- Adds the new song to the beginning of the linked list.

Save Database (saveDatabase):

- Writes the song database to a file.

List All Songs (listall):

- Displays details of all songs in the linked list.

Search and Display (search):

- Takes user input for a keyword.
- Searches for songs with matching title, artist, album, or genre.
- Displays matching songs.

Delete Song (del):

- Takes user input for the title of the song to delete.
- Searches for the song and deletes it from the linked list.

Free Database (freeDatabase):

- Frees the memory allocated for the entire song database when the program exits.

5. List of Functions:

struct song *loadDatabase():

- Input Parameters: None. Function relies on user input.
- Returned Result: Returns a pointer to the head of the linked list to get ready for next song.

void addSong(struct song **database):

- Input Parameters: Passing a double pointer, can modify both the content of the data it points to and the pointer itself.
- Returned Result: None. It performs actions.

void saveDatabase(struct song *database):

- Input Parameters: Points to the head of the linked list. Passing a single pointer, can modify the content of the data it points to but not the pointer itself.
- Returned Result: None. It performs actions.

void listAll(const struct song *database):

- Input Parameters: Pointer to the head of the linked list.

- Returned Result: None. It performs actions.

void search(const struct song *database):

- Input Parameters: Pointer to the head of the linked list.
- Returned Result: None. It performs actions.

void deleteSong(struct song **database):

- Input Parameters: Pointer to the head of the linked list and can modify pointer itself.
- Returned Result: None. It performs actions.

void freeDatabase(struct song *database):

- Input Parameters: Pointer to the head of the linked list.
- Returned Result: None. It performs actions.

6. Summary:

The program serves as a simple Song Database Management System using a linked list. It provides basic functionality for managing songs, with user-friendly text-based interactions. The linked list efficiently handles dynamic song entries.

Testing Document

1. Test Case: Add a Song to the Database

Input:

- Choose option 1 to add a new song.
- Enter valid information for a new song when prompted.

Expected Output:

- Song added successfully.
- Verify the added song by listing all songs (option 2) and checking if the added song is present.

2. Test Case: List All Songs

Input:

- Choose option 2 to list all songs.

Expected Output:

- Display a list of all songs in the database.

3. Test Case: Search for a Song

Input:

- Choose option 3 to search for a song.
- Enter a valid keyword (e.g., artist name, album title) to search.

Expected Output:

- Display information about the song(s) matching the search criteria.
- If no match is found, display a message indicating that.

4. Test Case: Delete a Song

Input:

- Choose option 4 to delete a song.
- Enter the title of an existing song to delete.

Expected Output:

- Display a success message if the song is deleted.
- Verify the song is no longer listed in the database.

5. Test Case: Save and Exit

Input:

- Choose option 5 to save and exit.

Expected Output:

- Save the current state of the database to a file.
- Free the allocated memory.

- Display an exit message.

6. Test Case: Load Database from File

Input:

- Run the program and choose to load the database from the file.

Expected Output:

- The program should load the existing database from the file.
- Verify the loaded songs by listing all songs (option 2).

7. Test Case: Invalid Choice

Input:

- Enter an invalid choice (e.g., a character or number outside the menu options).

Expected Output:

- Display a message indicating that the choice is invalid.
- Prompt the user to enter a valid option.

8. Test Case: Handling Empty Database File

Input:

- Create an empty database file.
- Run the program and choose to load the database from the file.

Expected Output:

- The program should handle an empty database file gracefully.

9. Test Case: Handling Incorrect Input

Input:

- Enter incorrect or unexpected input during song entry (e.g., entering a string for the released year).

Expected Output:

- The program should handle incorrect input gracefully and prompt the user to enter valid information.

10. Test Case: Deleting Non-existent Song

Input:

- Choose option 4 to delete a song.
- Enter the title of a non-existent song.

Expected Output:

- Display a message indicating that the song was not found.
- Prompt the user to enter a valid song title.